

United International University

CSE 3422: Software Engineering Lab

SELENIUM

11/05/26

Contents

1 Lab Overview **1**

2 Learning Objectives **1**

3 What is Selenium? **1**

4 Step 1: Install Python **1**

5 Step 2: Install Selenium **2**

6 Step 3: Install Browser Driver **2**

7 Step 4: First Selenium Program **2**

8 Code Explanation **3**

9 Browser Navigation **4**

10 Step 5: Interacting with Web Elements **4**

11 Locating Web Elements **4**

11.1 ID Selector 4

11.2 NAME Selector 5

11.3 CLASS NAME Selector 5

11.4 XPATH Selector 5

11.5 CSS Selector 5

12 Sending Data to Input Fields **6**

13 Clicking Buttons **6**

14 Clearing Fields **6**

15 Dropdown Selection **6**

16 Reading Text from Components **6**

17 Assertions and Testing **7**

18 Step 6: Extracting Multiple Elements **7**

19 Difference Between find_element and find_elements **8**

20 Complete Example: Simple Automated Test Case **8**

21 Understanding the Test **10**

22 Expected Output **11**

23 Why Automated Testing is Important **11**

24 Common Selenium Problems **11**

24.1 Element Not Found 11

24.2 Solution 11

25 Mini Testing Tutorial **12**

26 Lab Tasks **12**

27 Submission Requirements **13**

28 Conclusion **13**

1 Lab Overview

In this lab, students will learn how to use Selenium for browser automation and web testing using Python.

Students will learn:

- How to install Selenium
- How to configure browser drivers
- How to automate browser operations
- How to locate and interact with web elements
- How to perform basic web testing
- How to validate outputs using assertions

2 Learning Objectives

After completing this lab, students should be able to:

- Launch and control a browser using Selenium
- Navigate websites automatically
- Locate HTML components using different selectors
- Input data into forms
- Click buttons and links
- Read text from webpages
- Perform simple automated testing

3 What is Selenium?

Selenium is a browser automation framework used for:

- Web testing
- Form automation
- Web scraping
- Repetitive browser tasks

Selenium simulates real user interactions with a web browser.

4 Step 1: Install Python

Verify Python installation:

```
python --version
```

If Python is not installed, download from:

`https://python.org`

5 Step 2: Install Selenium

Install Selenium using pip:

```
pip install selenium
```

Verify installation:

```
pip show selenium
```

6 Step 3: Install Browser Driver

Selenium requires a browser driver.

Examples:

- Chrome → ChromeDriver
- Edge → EdgeDriver
- Firefox → GeckoDriver

In this lab we will use Microsoft Edge.

Download EdgeDriver from:

`https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/`

Ensure driver version matches browser version.

7 Step 4: First Selenium Program

Create a file named:

`test1.py`

```
from selenium import webdriver
from selenium.webdriver.edge.service import Service

# Keep browser open
options = webdriver.EdgeOptions()
options.add_experimental_option("detach", True)

service_obj = Service()

driver = webdriver.Edge(
    options=options,
    service=service_obj
)

```

```
driver.maximize_window()

driver.get("http://www.google.com")

print(driver.title)
print(driver.current_url)

driver.close()
```

8 Code Explanation

Import Selenium

```
from selenium import webdriver
```

Imports Selenium browser automation tools.

Create Browser Driver

```
driver = webdriver.Edge()
```

Launches Microsoft Edge browser.

Open Website

```
driver.get("http://www.google.com")
```

Navigates to the given website.

Browser Information

```
print(driver.title)
```

Prints website title.

```
print(driver.current_url)
```

Prints current URL.

9 Browser Navigation

```
driver.back()
driver.forward()
driver.refresh()
```

- back() → Go to previous page
- forward() → Go to next page
- refresh() → Reload page

10 Step 5: Interacting with Web Elements

Create a new file:

test2.py

```
from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By

options = webdriver.EdgeOptions()
options.add_experimental_option("detach", True)

service_obj = Service()

driver = webdriver.Edge(
    options=options,
    service=service_obj
)

driver.maximize_window()

driver.get(
"http://rahulshettyacademy.com/angularpractice/"
)
```

11 Locating Web Elements

Selenium locates components using selectors.

11.1 ID Selector

```
driver.find_element(
By.ID,
"exampleInputPassword1"
)
```

11.2 NAME Selector

```
driver.find_element(
By.NAME,
"email"
)
```

11.3 CLASS NAME Selector

```
driver.find_element(
By.CLASS_NAME,
"form-control"
)
```

11.4 XPATH Selector

Syntax:

```
//tagname[@attribute='value']
```

Example:

```
driver.find_element(
By.XPATH,
"//input[@type='password']"
)
```

11.5 CSS Selector

Syntax:

```
tagname[attribute='value']
```

Example:

```
driver.find_element(
By.CSS_SELECTOR,
"input[type='password']"
)
```

12 Sending Data to Input Fields

```
driver.find_element(
By.CLASS_NAME,
"form-control"
).send_keys("John")
```

13 Clicking Buttons

```
driver.find_element(
By.CLASS_NAME,
"btn"
).click()
```

14 Clearing Fields

```
driver.find_element(
By.ID,
"exampleInputPassword1"
).clear()
```

15 Dropdown Selection

```
driver.find_element(
By.ID,
"exampleFormControlSelect1"
).send_keys("Female")
```

16 Reading Text from Components

```
msg = driver.find_element(
By.CLASS_NAME,
"alert"
).text

print(msg)
```

17 Assertions and Testing

Assertions are used to validate expected output.

```
assert "Success" in msg
```

Meaning:

- If "Success" exists → Test passes
- Otherwise → Test fails

18 Step 6: Extracting Multiple Elements

Create:

test3.py

```
from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By
import time

options = webdriver.EdgeOptions()
options.add_experimental_option("detach", True)

service_obj = Service()

driver = webdriver.Edge(
    options=options,
    service=service_obj
)

driver.get("http://www.uiu.ac.bd")

driver.find_element(
By.LINK_TEXT,
"Important Contact"
).click()

time.sleep(5)

elements = driver.find_elements(
By.CLASS_NAME,
"section-wrapper"
)

for E in elements:
    print(E.text)
```

19 Difference Between find_element and find_elements

- find_element() → Returns one element
- find_elements() → Returns multiple elements

20 Complete Example: Simple Automated Test Case

Scenario

We want to test whether a user can successfully submit a form.

The test should verify:

- User can enter data
- User can click submit
- Success message appears

Test Website

We will use:

`https://rahulshettyacademy.com/angularpractice/`

Test Case Steps

1. Open website
2. Enter student name
3. Enter email
4. Enter password
5. Select gender
6. Click submit button
7. Verify success message

Automated Testing Code

Create a file:

`form_test.py`

```
from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By
import time

# Edge browser setup
```

```
options = webdriver.EdgeOptions()
options.add_experimental_option("detach", True)

service_obj = Service()

driver = webdriver.Edge(
    options=options,
    service=service_obj
)

# Open website
driver.get(
"https://rahulshettyacademy.com/angularpractice/"
)

driver.maximize_window()

# Enter name
driver.find_element(
By.NAME,
"name"
).send_keys("Sajid")

# Enter email
driver.find_element(
By.NAME,
"email"
).send_keys("sajid@test.com")

# Enter password
driver.find_element(
By.ID,
"exampleInputPassword1"
).send_keys("12345")

# Click checkbox
driver.find_element(
By.ID,
"exampleCheck1"
).click()

# Select gender
driver.find_element(
By.ID,
"exampleFormControlSelect1"
).send_keys("Male")

# Click submit button
```

```
driver.find_element(
By.CLASS_NAME,
"btn-success"
).click()

# Read success message
message = driver.find_element(
By.CLASS_NAME,
"alert-success"
).text

print("System Message:")
print(message)

# Assertion (Testing)
assert "Success" in message

print("TEST PASSED")

time.sleep(5)

driver.close()
```

21 Understanding the Test

Input Phase

The script enters user data into the form fields.

Action Phase

The script clicks:

- Checkbox
- Submit button

Validation Phase

The script checks whether:

`Success`

exists in the displayed message.

Assertion

```
assert "Success" in message
```

Meaning:

- If the message contains "Success" → Test passes
- Otherwise → Program throws an error

22 Expected Output

Terminal:

```
System Message:
Success! The Form has been submitted successfully!

TEST PASSED
```

23 Why Automated Testing is Important

Automated testing helps developers:

- Detect bugs quickly
- Test repeatedly without manual effort
- Validate software automatically
- Save development time

24 Common Selenium Problems

24.1 Element Not Found

Possible causes:

- Wrong selector
- Page not fully loaded
- Typo in selector

24.2 Solution

Use delays:

```
time.sleep(5)
```

25 Mini Testing Tutorial

Scenario

We want to test whether form submission works correctly.

Steps

1. Open website
2. Fill form
3. Click submit
4. Capture success message
5. Verify success text

Expected Result

System should display:

```
Success!
```

26 Lab Tasks

Task 1

Open Google using Selenium and print:

- Website title
- Current URL

Task 2

Open the practice form website and:

- Enter name
- Enter email
- Enter password
- Submit form

Task 3

Read the success message and verify using assertion.

Task 4

Visit a university website and print all text from a selected section.

27 Submission Requirements

Students must submit:

- Python source files
- Screenshots of successful execution
- Output screenshots

28 Conclusion

In this lab, students learned:

- Browser automation
- Web element interaction
- Automated testing basics
- Assertions and validation